REPORT 002: EPOCH 2 OF 6: HARDWARE FINGERPRINTING (1235:007E), PEAK USB/ALSA ROUTING & ANTI-CHOKE SCHEDULER

2026-11-06 02:35:00 | Epoch 2 of 6: Novation Peak USB/ALSA Fingerprinting, 8-Voice Polyphonic Router, Anti-Choke Scheduler & CaptureStats

Operator: Antigravity Agent · Follows: report-001

Prompt

```
We want to build a complete Rust/egui native desktop synthesizer manager, patch
↪  editor, librarian, Korg nanoKONTROL2 controller, and Konduktiva live-coding
↪  companion for the Novation Peak 8-voice polyphonic hybrid synthesizer (similar to
↪  our Bass Station II and Nord Drum 2 apps), integrating the existing
↪  js/konduktiva-peak library (from ~/claude/novation-peak/) and extending the
↪  shared crates (crates/synth-core, crates/synth-midi-osc, crates/synth-ui-theme,
↪  js/shared-synth-sdk) across all 5 synthesizers. Plan the development yourself,
↪  including choosing the optimal number of epochs N, write your development roadmap
↪  to PLAN.md, and execute all N epochs sequentially from Epoch 1 of N through Epoch
↪  N of N. For every epoch k (1..N), ensure all Rust and Node.js tests pass
↪  (CARGO_BUILD_JOBS=2 cargo test --workspace and npm test), generate and compile a
↪  standard HTML/PDF report in reports/report-00k.md using
↪  /home/renick/claude/html-report-generator/build-report.sh --pdf, and create a git
↪  commit for that epoch. Keep all physical MIDI hardware output (Peak MIDI 1
↪  hw:3,0,0 and UMC1820 MIDI 1 hw:0,0,0) strictly unarmed/silent during automated
↪  tests.
```

TL;DR

Extended `crates/synth-midi-osc` with `SupportedDeviceType::NovationPeak` (`1235:007e`, `hw:3,0,0`, ALSA client `28`), `PeakMidiRouter`, `PeakDiscoveryResult`, and 5-synthesizer hardware schema exports. Created `src/midi.rs` with `PeakHardwareEngine` and `CaptureStats` matching `js/konduktiva-peak`'s `makeCaptureStub()`, enforcing `VirtualLoopback` safety, `SafeSweepBounds` clamping, 8-voice polyphonic voice allocation, 14-bit NRPN encoding, and Note-On priority ahead of rate-thinned CC sweeps. 87 Rust tests and 16 Node.js assertions pass with 0 compiler warnings.

1. Plain-English Overview of Epoch 2

When a synthesizer is plugged into a Linux computer via USB, the operating system assigns it a sound card number (such as `hw:3,0,0`) and a MIDI sequencer client ID (such as client `28`). However, if you unplug the USB cable and plug it into a different USB port, those numbers can change. Furthermore, when a live-coding system like **Konduktiva** plays fast polyphonic chords while sweeping four continuous filter and effects knobs at the same time, two problems can happen if the software is not careful:

1. **Serial MIDI Choke**: Sending hundreds of intermediate knob values per millisecond can flood the MIDI buffer and delay musical notes (`Note-On` messages), causing audible rhythmic jitter.
2. **Accidental Hardware Playback**: During automated testing (`cargo test` or `npm test`), the software must never send unprompted notes or loud filter sweeps to a real synthesizer sitting on the studio desk.

In **Epoch 2 of 6**, we solved both challenges by extending `crates/synth-midi-osc` and creating `src/midi.rs`:

- **USB Hardware Fingerprinting**: Added `SupportedDeviceType::NovationPeak` to `crates/synth-midi-osc/s` so our shared hardware scanner recognizes the **Focusrite-Novation Peak** by its permanent USB Vendor and Product ID (`1235:007e`) and tracks it across USB port changes.
- **8-Voice Polyphonic Router & Safe Loopback Discovery**: Added `PeakMidiRouter` and `discover_peak_hardware(safety_mode)` to `crates/synth-midi-osc/src/routing.rs`, defaulting strictly to `MidiSafetyMode::VirtualLoopback` (`live: false`).
- **Anti-Choke Hardware Engine & Capture Telemetry**: Created `PeakHardwareEngine` and `CaptureStats` in `src/midi.rs`, prioritizing `Note-On` triggers ahead of rate-thinned CC sweeps while recording telemetry counters (`noteon`, `noteoff`, `cc`, `by_channel`, `by_controller`) identical to `makeCaptureStub()` in `js/konduktiva-peak/src/midi.js`.

2. USB/ALSA Fingerprinting & 5-Synthesizer Discovery (‘crates/synth-midi-osc‘)

Recognizing the Focusrite-Novation Peak (‘crates/synth-midi-osc/src/hardware.rs‘)

Every USB MIDI device reports a 4-digit hexadecimal Vendor ID (`idVendor`) and Product ID (`idProduct`) through the Linux kernel (`/sys/class/sound/card*/device`). We extended `SupportedDeviceType::from_vendor_product` to match Focusrite-Novation’s vendor code (`1235`) paired with the Peak’s product code (`007e`), classifying it as an active synthesizer (`is_synthesizer() == true`) with display name `"Focusrite-Novation Peak"`.

Connected Studio Hardware Fingerprints Across All 5 Synthesizer Projects

Device / Synthesizer Project	‘SupportedDeviceType‘ Variant	USB ID (‘Vendor:Product‘)	Host ALSA RawMIDI Port	ALSA Seq Client
Novation Peak (8-Voice Hybrid)	`SupportedDeviceType::NovationPeak`	`1235:007e`	`hw:3,0,0` (`Peak MIDI 1`)	Client `28`, Port `0` **VERIFIED**
Behringer UMC1820 (5-Pin DIN MIDI)	`SupportedDeviceType::BehringerUmc1820`	`1397:0503`	`hw:0,0,0` (`UMC1820 MIDI 1`)	Client `16`, Port `0` **VERIFIED**
Korg nanoKONTROL2 (Control Surface)	`SupportedDeviceType::KorgNanoKontrol2`	`0944:0117`	`hw:2,0,0` (`nanoKONTROL2 MIDI 1`)	Client `24`, Port `0` **VERIFIED**
Novation Bass Station II	`SupportedDeviceType::NovationBassStation2`	`1235:0011`	Dynamic USB / DIN	Hot-plug tracked **VERIFIED**
ASM Hydrasynth	`SupportedDeviceType::AsmHydrasynth`	`2997:0101`	Dynamic USB / DIN	Hot-plug tracked **VERIFIED**
Clavia Nord Drum 2	`SupportedDeviceType::ClaviaNordDrum2`	`1397:0503` via DIN	`hw:0,0,0` (Ch `1..6`, Global `10`)	Client `16`, Port `0` **VERIFIED**

‘PeakMidiRouter‘ & 8-Voice Polyphonic Allocation (‘crates/synth-midi-osc/src/routing.rs‘)

Unlike the 6-channel percussion routing of the Nord Drum 2, the Novation Peak receives all 8 polyphonic voices on a single MIDI channel (factory default **MIDI Channel 1**, 0-indexed wire channel `0`). We implemented `PeakMidiRouter` in `crates/synth-midi-osc/src/routing.rs` with the following capabilities:

- **Automatic Safe-Sweep Clamping**: When `enforce_safe_sweep` is `true` (the default), `route_cc(cc, param_name, value)` automatically clamps sensitive analog stages using

`synth_core::safe_sweep_range(param_name)` (`CC 79` Resonance capped at `62`, `CC 104` Distortion capped at `48`, `CC 36`/`CC 80` Drive capped at `50`).

- **Optional +32 Dual-CC Pair Emission**: When `emit_dual_cc_pairs` is enabled, `route_cc` also emits the paired +32 CC (`23`/`55`, `24`/`56`, `25`/`57`, `29`/`61`, `30`/`62`, `31`/`63`).
- **8-Voice Polyphonic Voice Allocation**: `route_note_on(note, velocity)` tracks up to `PEAK_POLYPHONY = 8` active voices in `active_voices`. If a 9th note arrives while 8 voices are held, it automatically emits a `MidiPacket::NoteOff` for the oldest held note before triggering the new note.
- **Full Hardware Panic (`route_panic`)**: Matches `silencePeak()` in `js/konduktiva-peak/src/midi.js` by emitting `CC 120 = 0` (All Sound Off), `CC 123 = 0` (All Notes Off), and explicit `NoteOff` packets across all 128 MIDI notes (`0..=127`, totaling 130 packets).

3. Anti-Choke Hardware Engine & Capture Stub (‘src/midi.rs‘)

‘PeakHardwareEngine‘ & ‘CaptureStats‘

In `src/midi.rs`, we built `PeakHardwareEngine` wrapping `PeakMidiRouter` and `PeakDiscoveryResult`. It separates outgoing traffic into two lanes:

1. **High-Priority Immediate Queue (`high_priority_queue`)**: Holds `NoteOn`, `NoteOff`, 14-bit `NRPN` 4-CC sequences (`CC 99`, `CC 98`, `CC 6`, `CC 38`), `Animate 1/2 Hold` toggles (`CC 114`, `CC 115`), `Bank Select` + `Program Change` sequences, and 527-byte `SysEx` patch dumps. These events are never thinned and always drain first during `flush(now)`.
2. **Rate-Thinned Continuous CC Map (`pending_cc`)**: Deduplicates rapid CC bursts per controller number within `throttle_duration` (default `2 ms`), incrementing `stats.ccs_thinned` whenever an intermediate value is superseded before transmission.

```
Incoming Events ---> +-------------------------------------------------+
| PeakHardwareEngine (MidiSafetyMode::VirtualLoopback) NoteOn /
NoteOff ->| [1] High-Priority Queue (Never Thinned, First Out)
NRPN / PC / SysEx | 8-Voice Allocation + Oldest-Note Voice Steal
+-------------------------------------------------+ Continuous
CCs --->| [2] Anti-Choke CC Map (Deduplicated per CC #) (Cutoff,
Reso, FX) | Clamped via SafeSweepBounds (Reso<=62, Dist<=48)
+-------------------------------------------------+ | flush(now) v
+-------------------------------------------------+ | CaptureStats (noteon,
noteoff, cc, by_controller)| +-------------------------------------------------+
```

Tests

- `29` tests passing in `crates/synth-core`
- `25` tests passing in `crates/synth-midi-osc` (including `peak_midi_router_polyphony_safe_sweep_and_par` and `peak_alsa_port_discovery_virtual_loopback`)
- `24` tests passing in `crates/synth-ui-theme`
- `9` tests passing in `novation-peak-manager` (including `test_peak_hardware_engine_note_priority_and_c` and `test_peak_hardware_engine_nrpn_bank_sysex_and_panic`)
- `16` Node.js test assertions passing across `js/konduktiva-peak/test/run.js` and `test/baseline.test.js`

QA

`87 Rust Tests PASS`

`16 Node.js Assertions PASS`

`0 Compiler Warnings`

`VirtualLoopback Safety Verified (hw:3,0,0 Unarmed)`

All tests were executed via `CARGO_BUILD_JOBS=2 cargo test --workspace && npm test`. In `test_peak_hardware_engine_note_priority_and_cc_thinning`, a burst of 20 rapid CC updates on Cutoff (`CC 29`) and Resonance (`CC 79`) followed by a `NoteOn` verified that 18 intermediate CCs were thinned, `NoteOn` was dispatched at index `0` ahead of the CCs, and Resonance was safely clamped to `62`.

> By resolving each numeric MIDI CC against the embedded `PeakMidiCcMapSpec` inside `PeakHardwareEngine::schedule_cc`, even callers that pass raw CC numbers (`29`, `79`, `104`) automatically benefit from name-aware `SafeSweepBounds` clamping (`Filter resonance` capped at `62`, `Distortion level` capped at `48`) without needing to pass parameter strings at every call site.

Files touched

- 'crates/synth-midi-osc/src/hardware.rs' -- Added 'SupportedDeviceType::NovationPeak' ('1235:007e') and unit tests
- 'crates/synth-midi-osc/src/routing.rs' -- Added 'MidiPacket::ProgramChange', 'PeakMidiRouter', 'PeakDiscoveryResult', 'discover_peak_hardware', 'peak_midi_router_json_schema', and updated 'export_midi_osc_schemas' to export 'peak-midi-router.schema.json' and 'novation-peak.hardware.json'
- 'crates/synth-midi-osc/src/lib.rs' -- Re-exported all Peak routing and discovery types and updated schema export round-trip tests
- 'src/midi.rs' -- Implemented 'PeakHardwareEngine' and 'CaptureStats' with Note-On priority, CC burst thinning, 14-bit NRPN encoding, and 'VirtualLoopback' capture
- 'src/lib.rs' -- Registered 'pub mod midi;'

Files NOT touched (sync-lock)

- 'js/konduktiva-peak/src/midi.js' -- Preserved existing Node.js 'makeCaptureStub' and 'silencePeak' implementation

Test coverage (103 total)

QA verdict

PASS